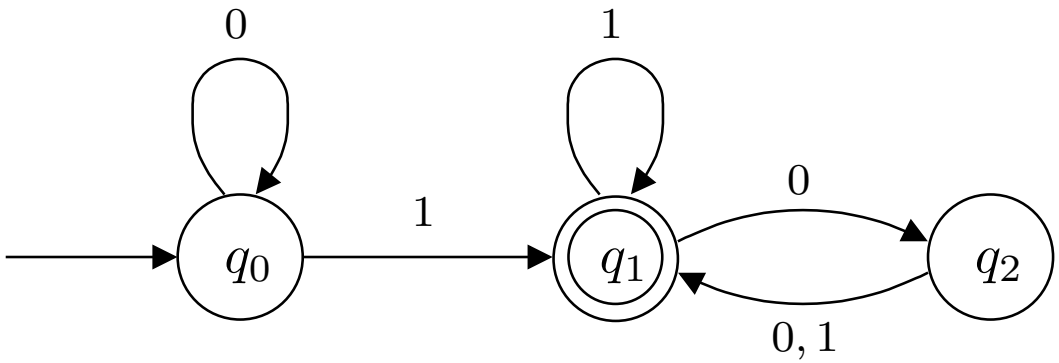
Exam 1 Review (Chapter 1)

Thumrongsak Kosiyatrakul
tkosiyat@cs.pitt.edu

Deterministic Finite Automaton (DFA)

- State Diagram of a DFA
- Formal Definition of a DFA $(Q, \Sigma, \delta, q_0, F)$
  - Q is a non-empty set of states
  - Σ is a finite set of symbols
  - $\delta : Q \times \Sigma \to Q$ transition function
  - $q_0 \in Q$ is the start state
  - F is a set of accept states

Formal Definition of a DFA



- $M_1 = (Q, \Sigma, \delta, q_0, F)$
  - $Q = \{q_0, q_1, q_2\}$
  - $\Sigma = \{0, 1\}$
  - δ can be defined using the table below:

δ	0	1
q_0	q_0	q_1
q_1	q_2	q_1
q_2	q_1	q_1

  - q_0 is the start state
  - $F = \{q_1\}$
- The state diagram and its formal definition are equivalent

Deterministic Finite Automaton (DFA)

- How a DFA process an input string?
  - Accept/Reject
- Language of a DFA
  - The set of all strings accepted by the DFA
- A language is **regular** is there are finite state machines that recognize it

Language Operators

- A language is a set of strings
- Given language A and B:
  - $A \cup B = \{x \mid x \in A \text{ or } x \in B\}$
  - $A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$
  - $A^* = \{x_1 x_2 \ldots x_k \mid k \geq 0 \text{ and } x_i \in A\}$
- Regular language is closed under union, concatenation, and star operations
  - Given two regular languages A and B
    - $A \cup B$ is a regular language
    - AB is a regular language
    - A^* and B^* are regular languages

Tool

- Given two DFA M_A and M_B
  - We obtain an algorithm to construct a DFA M such that

$$L(M) = L(M_A) \cup L(M_B)$$

  - The algorithm is defined based on formal definitions of DFAs M_A and M_B

$\mathbb{L}$ is regular under union operation

- Let M_A recognizes A, where $M_A = (Q_A, \Sigma, \delta_A, q_A, F_A)$
- Let M_B recognizes B, where $M_B = (Q_B, \Sigma, \delta_B, q_B, F_B)$
- Machine $M = (Q, \Sigma, \delta, q_0, F)$ that recognizes $A \cup B$ can be constructed as follows:
  1. $Q = \{(r_1, r_2) \mid r_1 \in Q_A \text{ and } r_2 \in Q_B\}$
  2. For each $(r_1, r_2) \in Q$ and $a \in \Sigma$

$$\delta((r_1, r_2), a) = (\delta_A(r_1, a), \delta_B(r_2, a))$$

  3. $q_0 = (q_A, q_B)$
  4. $F = \{(r_1, r_2) \mid r_1 \in F_A \text{ or } r_2 \in F_B\}$

Nondeterministic Finite Automaton (NFA)

- Can split into multiple copies of itself
- At a state, it may have 0, 1, or more than 1 exiting arrow for each symbol
- At a state, it may have 0, 1, or more than 1 exiting arrow for ε
- We generally use a computational tree to compute an NFA

Formal Definition of A Nondeterministic Finite Automaton

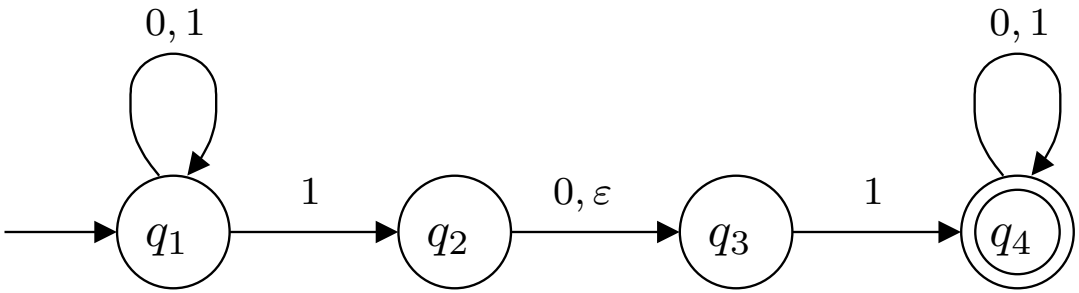
- A **nondeterministic finite automaton** is a 5-tuple

$$(Q, \Sigma, \delta, q0, F)$$

1. Q is a finite set of states
2. Σ is a finite alphabet
3. $\delta : Q \times \Sigma_\varepsilon \to \mathcal{P}(Q)$ is the transition function,
   - $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ and
   - $\mathcal{P}(Q)$ is the **powerset** of Q (set of set of states).
4. $q_0 \in Q$ is the start state
5. $F \subseteq Q$ is the set of accept states.

- **Notes**
  - In an NFA, one input symbol can change the state of the machine to multiple states.
    - Split to multiple copies with different current states
    - Example: $\delta(q_0, 1) = \{q_0, q_1\}$

Example



- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$ and $\Sigma_\varepsilon = \{0, 1, \varepsilon\}$
- δ is given as

δ	0	1	ε
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

  - We treat ε as a regular input symbol
  - If there is no ε transitions, we can ignore the ε column
- q_1 is the start state
- $F = \{q_4\}$

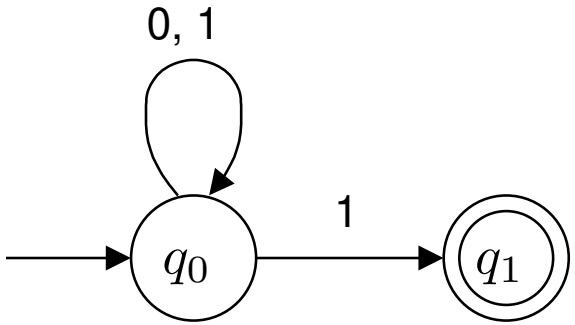
Equivalence of NFAs and DFAs

Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

- Let $N = (Q, \Sigma, \delta, q_0, F)$ be the NFA recognizing some language A
- We are going to construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$ recognizing A
- Let's consider the case where N has no ε transitions.
  1. $Q' = \mathcal{P}(Q)$
  2. $\delta'(R, a) = \bigcup_{r \in R} \delta(r, a)$
  3. $q_0' = \{q_0\}$
  4. $F' = \{R \in Q' \mid R \text{ contains an accept state of } N\}$

Example

- Let Σ be $\{0, 1\}$. The following NFA N recognizes the language A where A is a set of strings that end with a 1.



- $N = (Q, \Sigma, \delta, q_0, F)$
  1. $Q = \{q_0, q_1\}$,
  2. $\Sigma = \{0, 1\}$
  3. δ is given as

	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\emptyset$

  4. q_0 is the start state
  5. $F = \{q_1\}$

Example

- Construct a DFA $M = (Q', \Sigma, \delta', q_0', F')$
  - $Q' = \mathcal{P}(Q) = \mathcal{P}(\{q_0, q_1\})$

$$Q' = \{\emptyset, \{q_0\}, \{q_1\}, \{q_0, q_1\}\}$$

  - We will construct δ' later
  - $q_0' = \{q_0\}$ where q_0 is the start state of the NFA
  - $F' = \{R \in Q' \mid R \text{ contains an accept state of } N\}$

$$F' = \{\{q_1\}, \{q_0, q_1\}\}$$

where $F = \{q_1\}$

Example

- Let's focus on transition functions
- The transition function δ of the NFA is as follows:

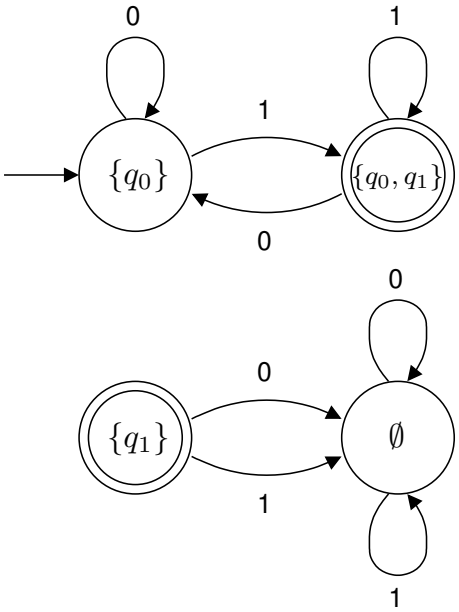
	0	1
q_0	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\emptyset$	$\emptyset$

- Recall that the set of state of the equivalent DFA is the power set of set of state of the NFA

δ'	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0\}$	$\{q_0, q_1\}$

Example

- The state diagram of the machine $M = (Q', \Sigma, \delta', q_0', F')$ equivalent to N ($L(M) = L(N)$) is shown below:



- Without bottom part, it is the same as one of our previous example
- It is okay for a DFA to have unreachable states

Regular Expression

- A regular expression expresses a regular language
- A regular expression can be constructed using these rules:
  1. If $a \in \Sigma$, a is a regular expression
  2. ε is a regular expression
  3. $\emptyset$ is a regular expression
  4. If R_1 and R_2 are regular expressions, $R_1 \cup R_2$ is a regular expression
  5. If R_1 and R_2 are regular expressions, $R_1 R_2$ is a regular expression
  6. If R_1 is a regular expression, R_1^* is a regular expression

Tool: Regular Expression to NFA

- Construct an NFA from a regular expression
  - Based on closure of union, concatenation, and star
- Given a regular expression
  1. Break it down into steps how to construct the regular expression
  2. For each step, slowly construct an NFA

Regular Expression to NFA

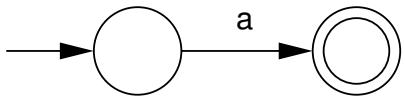
- Convert the regular expression $(ab \cup a)^*$ to an NFA.



Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

a



b

ab

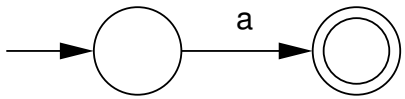
ab U a

(ab U a) *

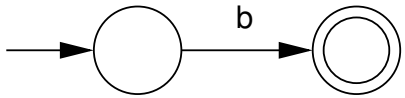
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

a



b



ab

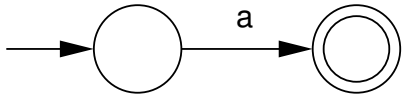
ab U a

(ab U a) *

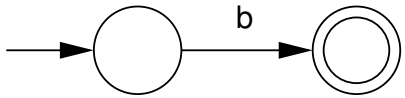
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

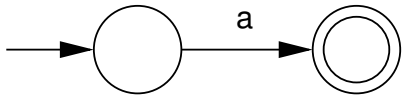
a



b



ab



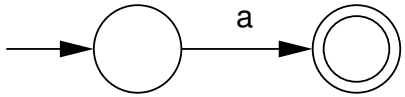
ab U a

(ab U a) *

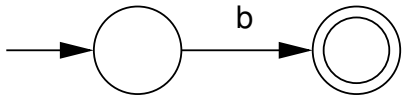
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

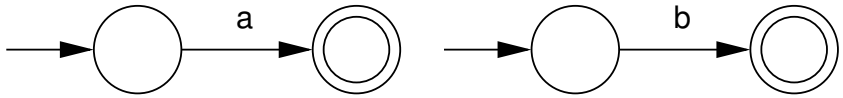
a



b



ab



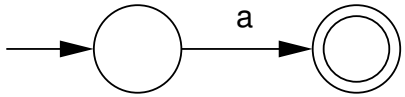
ab U a

(ab U a) *

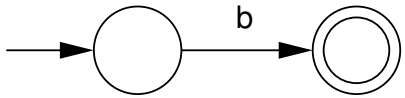
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

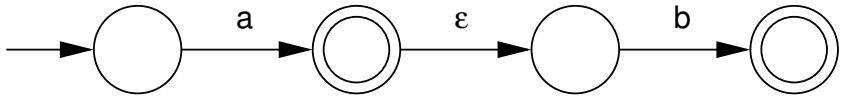
a



b



ab



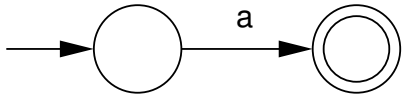
ab U a

(ab U a) *

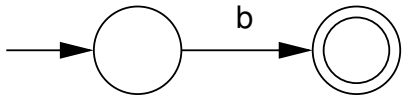
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

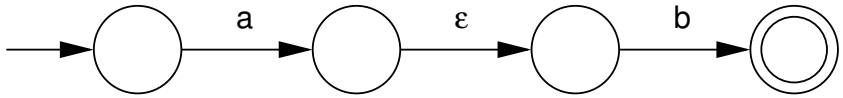
a



b



ab



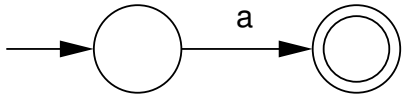
ab U a

(ab U a) *

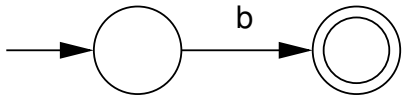
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

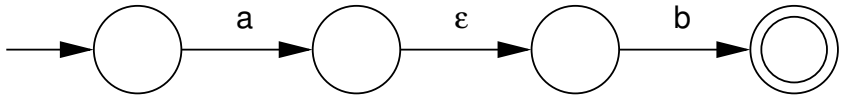
a



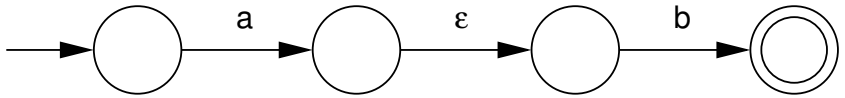
b



ab



ab U a

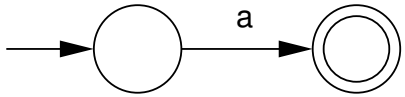


(ab U a) *

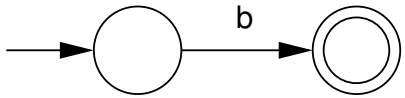
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

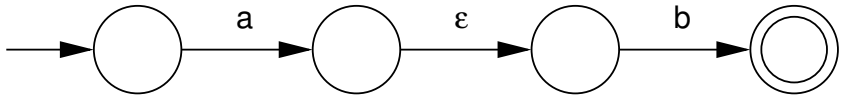
a



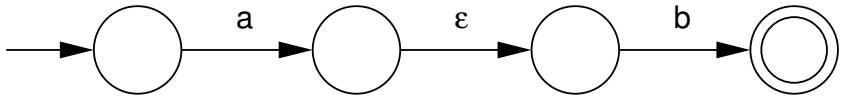
b

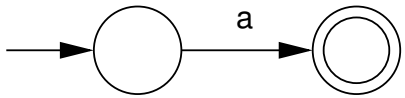


ab



ab U a





(ab U a) *

Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

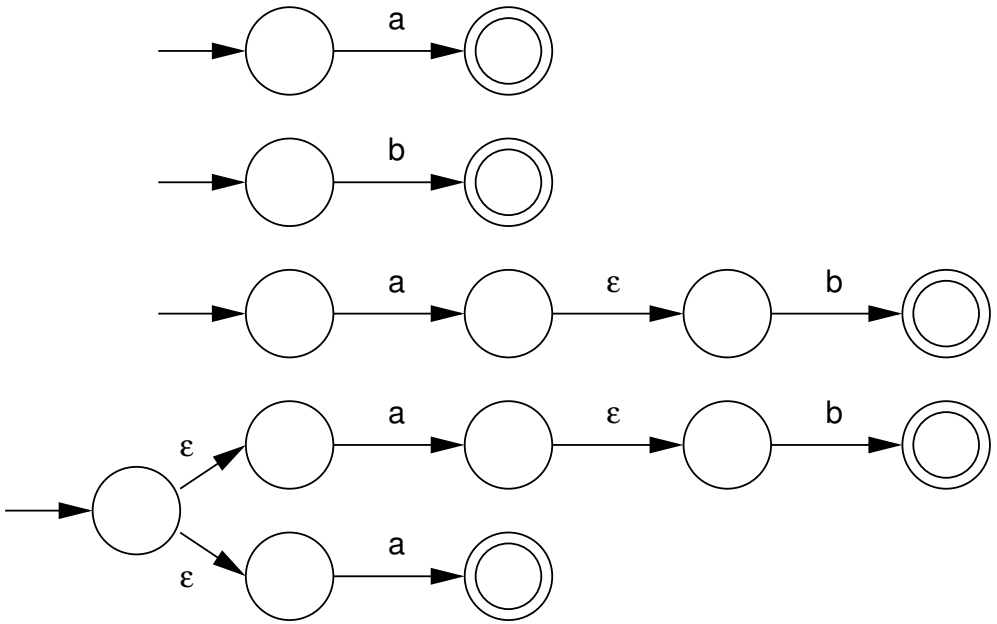
a

b

ab

ab U a

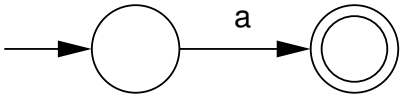
(ab U a) *



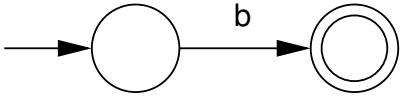
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

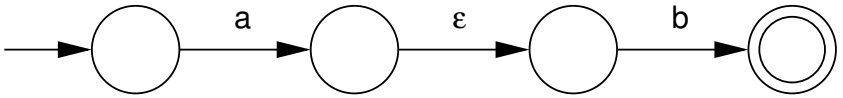
a



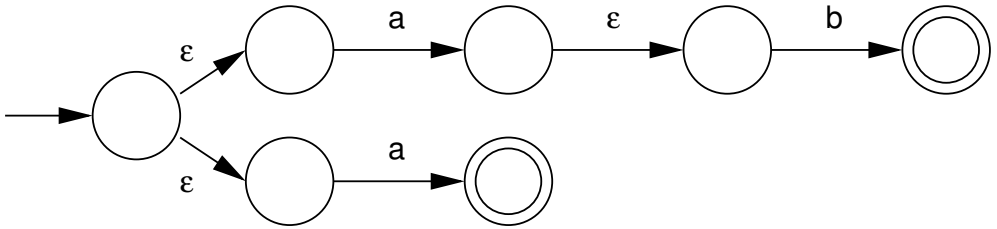
b



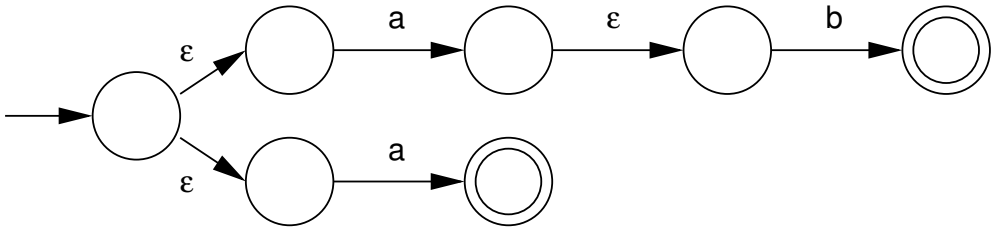
ab



ab U a

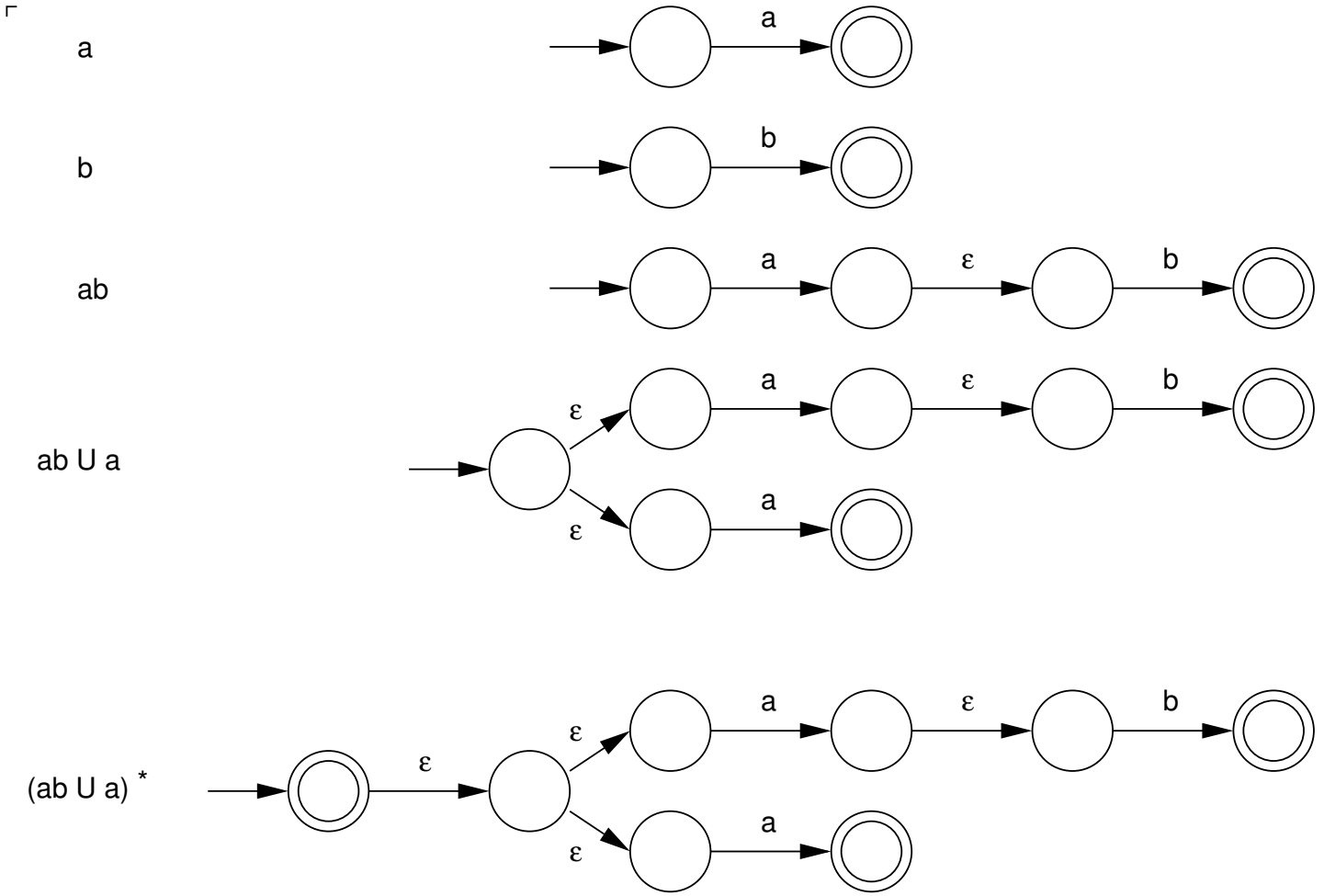


(ab U a) *



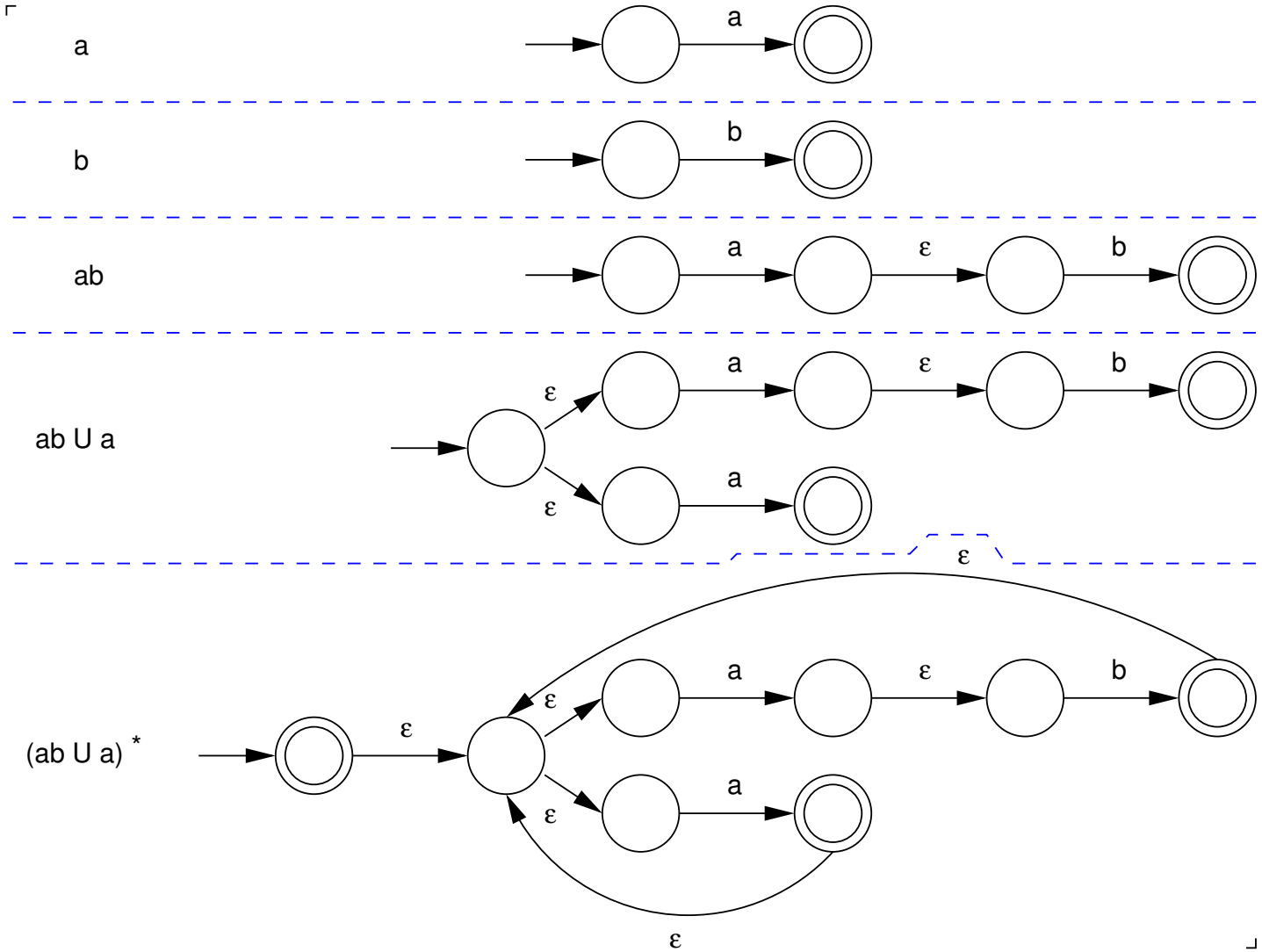
Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.



Regular Expression to NFA

- Convert the regular expression $(ab \cup a)^*$ to an NFA.

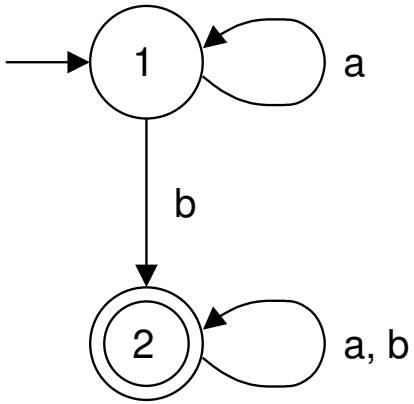


Regular Language

- Every regular language can be expressed by a regular expression
- Convert from an DFA to a regular expression
  1. Convert a given n-state DFA to $n + 2$ states (GNFA)
     - Add a new start state s with ε transition
     - Add a new accept state a with ε transitions
     - Convert all transition function from symbol to regular expression
  2. Get rid of original states of DFA without changing its language until we only have two states
  3. The label from s to a is a regular expression that expresses the language of a given DFA

Example

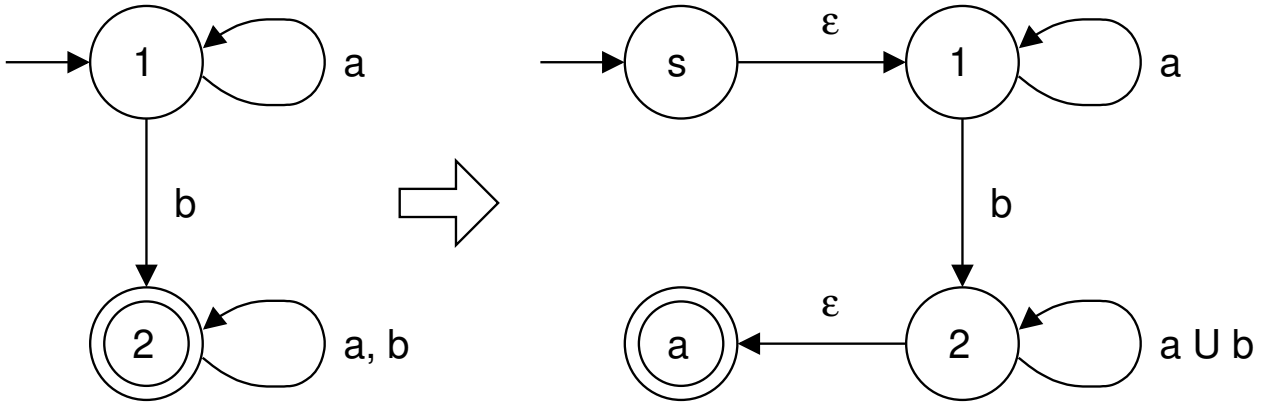
- DFA



$$\{w \mid w \text{ contains at least one } b\}$$

Example

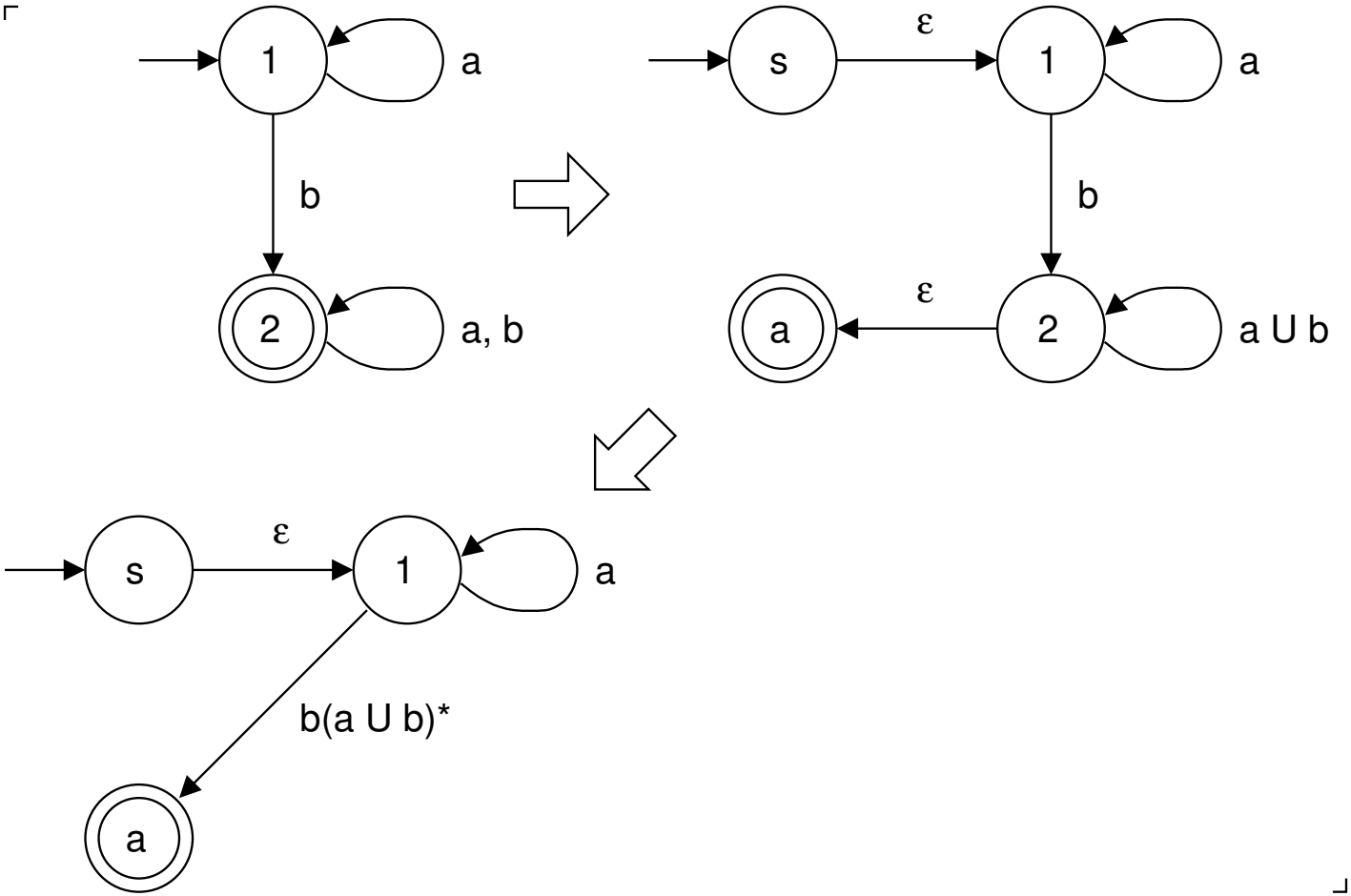
- DFA to GNFA



$$\{w \mid w \text{ contains at least one } b\}$$

Example

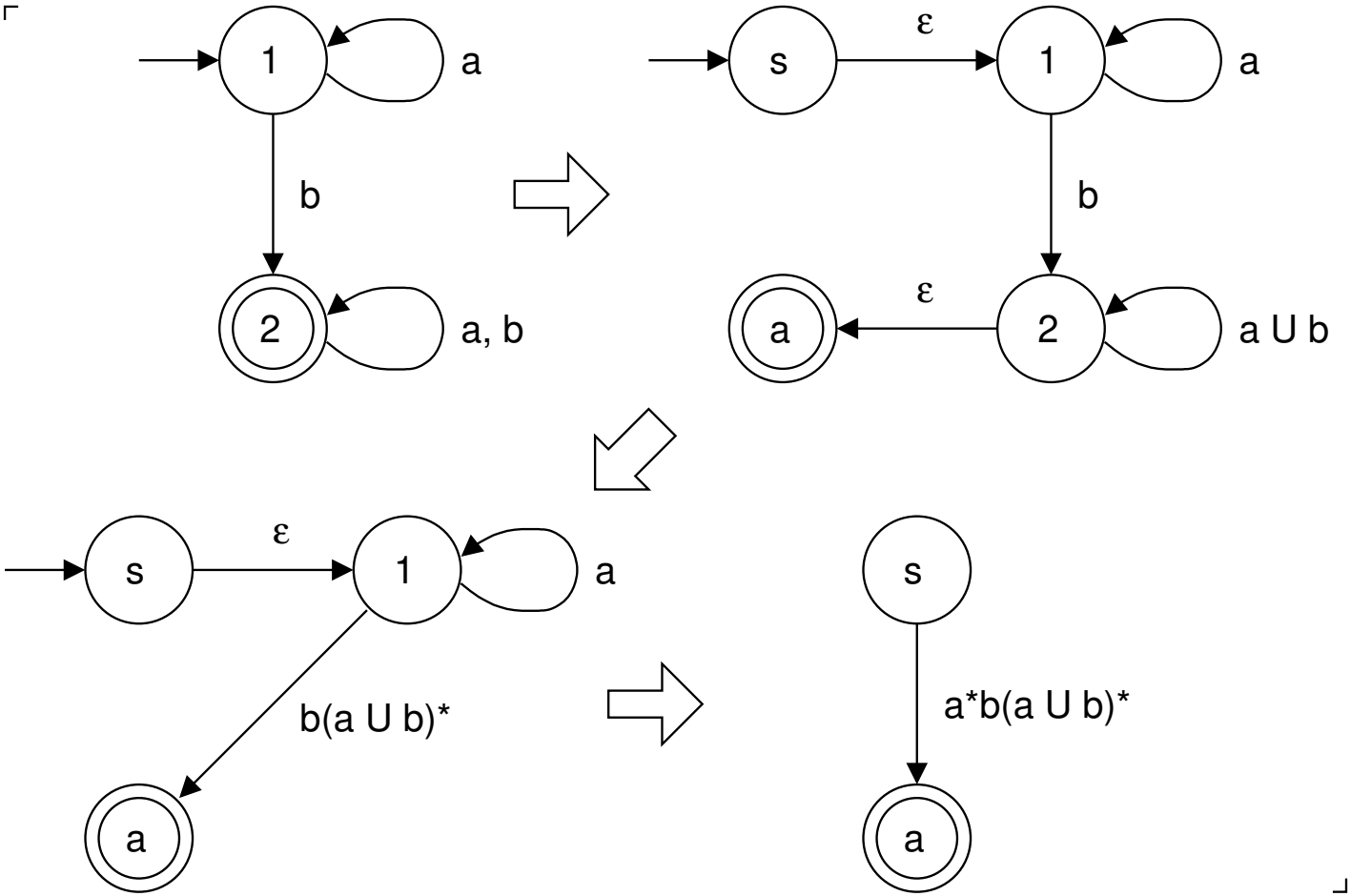
- Get rid of state 2



$$\{w \mid w \text{ contains at least one } b\}$$

Example

- Get rid of state 1



$\{w \mid w$ contains at least one $b\}$

Pumping Lemma

- The Pumping Lemma is based on a property of a regular language:

  If A is regular, for a string $s \in A$ of length at least P, s can be divided into $s = xyz$ satisfying the following conditions:

  1. $xy^iz \in A$ for any $i \geq 0$
  2. $|y| > 0$
  3. $|xy| \leq p$

- To use the Pumping lemma to prove that a language is not regular, we use prove by contradiction

Show that $A = \{a^i b^j \mid i = j \text{ or } 2i = j\}$

Assume that A is regular. Since A is regular, the Pumping lemma says that for any string $s \in A$ of length at least p, s can be divided into $s = xyz$ satisfying the following conditions:

1. $xy^i z \in D$ for any $i \geq 0$
2. $|y| > 0$
3. $|xy| \leq p$

Let $s = a^p b^{2p}$. In this case, $i = p$ and $j = 2p$. Since $2i = j$, $s \in A$. Since s starts with p as, to satisfy the third condition, x and y are strings that contain nothing but as. In other words, $x = a^q$ for any $q \geq 0$, and $y = a^r$ for any $r > 0$. Note that r must be greater than 0 because $|y| = |a^k| = r$, and the condition 2 says that $|y| > 0$. Since $x = a^q$ and $y = a^r$, $z = a^{p-(q+r)} b^{2p}$. Let $i = 0$. We have

$$\begin{aligned} xy^i z &= xy^0 z = xz \\ &= a^q a^{p-(q+r)} b^{2p} \\ &= a^{p-r} b^{2p} \end{aligned}$$

For the string $a^{p-r} b^{2p}$, $i = p - r$ and $j = 2p$. Thus, for this string to be in A, $p - r$ must be equal to $2p$ or $2(p - r)$ must be equal to $2p$. For $p - r$ to be equal to $2p$, r must be equal to $-p$. For $2(p - r)$ to be equal to $2p$, r must be 0. But r must be greater than 0. Thus, $a^{p-r} b^{2p} \notin A$ — contradiction. Therefore, A is not regular.

Note

- Let $\Sigma = \{0, 1\}$ and consider the following language:

$$B = \{ww \mid w \in \Sigma^*\}$$

- Suppose you pick the string s to be 0^p0^p
  - This string does not lead to a contradiction
  - We get the usual x, y, and z
    - $x = 0^j$ for $j \geq 0$
    - $y = 0^k$ for $k > 0$
    - $z = 0^{p-(j+k)}0^p$
  - $xy^iz = 0^j(0^k)^i0^{p-(j+k)}0^p = 0^{2p+k(i-1)}$
  - For any i, you can always find $k > 0$ such that $2p + k(i-1) = 2x$ for some positive integer x
    - This allows us to split $0^{2p+k(i-1)}$ to 0^x0^x which is in the language B
- You need to pick another s